QUIZ 1 NOTACION ASINTOTICA Y BIG O

Parte 1: Falso o Verdadero

Lee cada afirmación y marca con una 'X' si es Falsa (F) o Verdadera (V).

1. [F] (VF) La notación Big O se utiliza principalmente para describir el límite inferior (el mejor tiempo de ejecución posible) de un algoritmo.
2. [V] (VF) Si el tiempo de ejecución exacto de un algoritmo está dado por la función f(n) = 500n + 3, su complejidad asintótica se clasifica como O(n)
3. [F] (VF) Si se demuestra que un algoritmo realiza siempre 3n + 5 operaciones, es formalmente válido decir matemáticamente que su complejidad está acotada por O(n^3), aunque sea una cota superior muy holgada.

Parte 2: Selección Múltiple (Teoría y Conceptos)Selecciona la respuesta correcta.

4. ¿Cuál de las siguientes complejidades temporales representa un crecimiento más rápido (es decir, el algoritmo es más ineficiente para valores grandes de n)?

   a) O(1)

   b) O(log n)

   c) O(n)

   d) O(n^2)

5. ¿Cuál es la complejidad temporal asintótica en notación Big O de la función matemática f(n) = 4n^3 + 2n^2 log n + 1000?

   a) O(n^3)

   b) O(n^2 \log n)

   c) O(n^4)

   d) O(\log n)

6. **Tienes un arreglo (array) en memoria con 10,000 elementos. ¿Cuál de estas operaciones tiene una complejidad de O(1)?**

   a) Buscar un valor específico recorriendo el arreglo elemento por elemento.

   b) Acceder directamente al elemento ubicado en el índice 500.

   c) Imprimir en pantalla todos y cada uno de los elementos del arreglo.

   d) Ordenar los elementos del arreglo de menor a mayor valor.

7. **¿Cuál de los siguientes fragmentos de lógica de programación describe mejor un algoritmo con una complejidad de O(n)?**

   a) Una función matemática que recibe la variable n y retorna el cálculo de n * 2.

   b) Dos bucles 'for' anidados donde ambos iteran desde 0 hasta n-1.

   c) Un bucle 'while' que en cada iteración divide el valor actual de n a la mitad.

   d) Un bucle 'for' simple que itera desde 0 hasta n-1, realizando una suma en cada paso.

Parte 3: Análisis de Código Determina la complejidad Big O del peor caso para los siguientes fragmentos de código.

8. Analiza el siguiente pseudocódigo:

```
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            imprimir(i, j);
    }
}
```

a) O(n)
b) O(n^2)
c) O(log n)
d) O(1)

9. Determina la complejidad del siguiente algoritmo con bucles secuenciales:

```
    for (int i = 0; i < n; i++) {
        imprimir(i);
    }
    for (int j = 0; j < n; j++) {
        imprimir(j);
}
```

a) O(n)
b) O(n^2)
c) O(2n^2)
d) O(\log n)

10. Analiza la complejidad del siguiente fragmento de iteración:

```
    int i = 1;
        while (i < n) {
            i = i * 2;
    imprimir(i);
    }
```

a) O(n)
b) O(1)
c) O(n log n)
d) O(log n)

11. **Implementación y diseño de algoritmos.**

Escribe una función (puedes usar C++ o pseudocódigo estructurado) que reciba como parámetros un arreglo de números enteros y el tamaño de dicho arreglo.

Tu objetivo es que la función cumpla con **dos requisitos**:

1. Debe imprimir en pantalla todas las combinaciones posibles de pares de números del arreglo (por ejemplo, si el arreglo es [1, 2], debe imprimir 1-1, 1-2, 2-1, 2-2).
2. El algoritmo debe tener una complejidad temporal asintótica exacta de **O(n^2)** en el peor de los casos.

Escribe tu código en el recuadro inferior y, al lado, explica brevemente en un par de líneas por qué la estructura que diseñaste garantiza una complejidad de O(n^2).

```
def imprimir_combinaciones(arr, n):
    for i in range(n):
        for j in range(n):
            print(f"{arr[i]}-{arr[j]}")
if __name__ == "__main__":
    ejemplo = [1, 2, 3]
    imprimir_combinaciones(ejemplo, len(ejemplo))
```

 # Uso dos bucles anidados que recorren el arreglo de 0 a n-1, eso hace que cada bucle realiza n iteraciones.
 # Por tanto se realizan exactamente (n * n) impresiones en el peor caso, garantizando O(n^2).